Restaurant Ordering System

COMP3006 – Assignment 2 - Report

Section 0 – Links

GitHub repository link: https://github.com/Plymouth-University/coursework-jackbadman

Demonstration video link:

Section 1 – Requirements

1.1 Problem Context

Many small, independent takeaway restaurants rely on phone-based or in-store point-of-sale systems that are primarily designed for face-to-face transactions rather than remote ordering. This often results in handwritten order notes, verbal confirmations, and manual re-entry of orders into kitchen systems. Such workflows increase the likelihood of transcription errors, missed items, and incorrect pricing, particularly during busy periods. Customers are typically given only estimated wait times, with limited visibility of order progress, while menu updates such as item availability or price changes are often communicated verbally or via static displays, leading to inconsistency and operational inefficiency.

1.2 Target Users

The system targets three primary user groups. **Customers** require a reliable way to browse menu items, place orders remotely, and track order progress without contacting the restaurant. **Restaurant staff** require an interface to view incoming orders and update their status efficiently during service. **Administrators** require secure access to manage menu categories, menu items, pricing, and availability to ensure the system reflects current offerings.

1.3 Functional Requirements

Functional requirements were defined according to user role. Customers must be able to browse menu categories and items retrieved from a backend database, with each item including a name, description, price, and category (FR1). Customers must also be able to place orders consisting of one or more menu items, with submitted orders stored persistently and assigned an initial status (FR2). Order status changes must be communicated in real time using WebSockets, allowing connected clients to observe updates without refreshing the interface (FR3).

Restaurant staff must be able to view active orders and update their status (e.g. received, preparing, ready), with changes reflected immediately across clients (FR4). Administrators must be able to create, update, and delete menu categories and items through authenticated interfaces to support dynamic menu management (FR5). The system must enforce authentication and restrict access to staff and administrative functionality based on user roles (FR6).

1.4 Requirements Prioritisation

Requirements were prioritised based on operational impact and module constraints. Core ordering functionality, persistent storage, and real-time updates were prioritised to ensure reliability during

peak usage and to satisfy distributed client–server interaction requirements. Administrative functionality and authentication were prioritised to support maintainability and access control, while non-essential features were deprioritised to ensure delivery of a stable minimum viable product within the available timeframe.

Section 2 – Design

2.1 System Architecture

The system adopts a monolithic client–server architecture, which is appropriate for the scale and scope of the application. The frontend is implemented as a React single-page application (SPA) and the backend as a Node.js and Express service, with each deployed in its own Docker container to ensure consistent runtime environments across development and testing. The frontend communicates with the backend via RESTful HTTP endpoints, while MongoDB is used as the persistence layer for users, menu items, orders, and order items. In addition to standard request–response communication, WebSockets are used to propagate real-time order status updates to connected clients.

Figure 1 presents the logical component diagram of the system. Although authentication, order handling, and menu management are shown as separate components, these represent logical modules within a single monolithic Express application, rather than independent microservices. This design simplifies deployment and reduces operational overhead while still maintaining clear separation of responsibilities within the codebase. The backend exposes REST endpoints through an Express router and uses JWT-based authentication middleware to secure protected routes. WebSockets enable the backend to push order status updates to customer and staff interfaces without requiring client polling.

2.2 Data and Code Structures

The system's data model is illustrated in **Figure 2** (ERD). Core entities include User, Category, MenuItem, Order, and OrderItem. Relationships are explicitly defined, with orders linked to users and order items linked to both orders and menu items. This structure supports data consistency while remaining flexible, which is well suited to MongoDB's document-oriented model. Constraints such as unique user emails, role-based access, and minimum order quantities are enforced at schema level using Mongoose.

At code level, the backend follows an MVC-inspired structure. Express routes handle HTTP concerns and delegate request processing to controllers, which encapsulate business logic such as order creation and validation. Mongoose models represent persistent data and enforce schema constraints. Authentication is handled through reusable middleware, allowing access control to be applied consistently across routes.

2.3 Design Practices

Several established design practices were applied. Separation of concerns is evident in the clear division between routing, controller logic, middleware, and data models, supporting maintainability and testability. The Single Responsibility Principle is applied by isolating authentication logic within JWT middleware and restricting controllers to a single domain responsibility (e.g. order handling). DRY principles are observed through reuse of shared middleware and validation logic across routes.

2.4 Interaction Design and Justification

Figure 3 presents a sequence diagram illustrating the interaction when a customer places an order. The diagram shows the customer interface submitting an authenticated request, JWT validation by middleware, order persistence in MongoDB, and emission of a WebSocket event to notify staff dashboards in real time. This interaction pattern justifies the use of WebSockets, as it enables timely updates without repeated client polling and supports the requirement for distributed client–server communication.

Overall, the chosen architecture and design structures are appropriate for the project requirements. The monolithic approach balances simplicity with modularity, the data model supports reliable order management, and the interaction design enables responsive, real-time behaviour while remaining maintainable and testable.

Section 3 – Testing

A multi-layered testing strategy was adopted to validate the correctness, robustness, and usability of the system. Testing was conducted at unit, integration, system, and usability levels using both automated and manual approaches, in line with the testing pyramid.

3.1 Automated Unit and Integration Testing

Automated testing was implemented using Jest and executed locally and within the continuous integration (CI) pipeline. Unit tests validated isolated components and business logic, allowing faults to be identified early and independently of the wider system. For example, authMiddleware.test.js (**Figure 4**) verifies authentication behaviour in isolation by mocking Express request and response objects. The test simulates unauthenticated requests to confirm a 401 response is returned and execution halts, and authenticated requests to ensure a valid JSON Web Token is decoded and user data is attached to the request before processing continues. This demonstrates unit testing of security-critical logic using mocking and coverage of both failure and success paths.

Additional unit tests were applied to controller and model logic. orderController.test.js and userController.test.js cover duplicate user registration, invalid order submissions, and successful order creation and retrieval. Model-level validation was tested in orderItem.model.test.js, ensuring schema constraints such as rejecting quantities less than one are enforced at the data layer.

Integration testing validated interactions between Express routes, authentication middleware, controllers, and the MongoDB data layer. The orders.api.test.js integration test (**Figure 5-6**) exercises the orders API end-to-end using Supertest and an isolated in-memory MongoDB instance. Authenticated HTTP requests are issued using a real JWT generated during test setup, verifying order creation, order retrieval with and without items, and rejection of unauthenticated access. Assertions against both HTTP responses and persisted database records confirm correct data flow and component interaction. A simple smoke test was included to verify correct test environment configuration.

3.2 Manual System and Usability Testing

Manual system testing verified end-to-end functionality across the frontend, backend, and database, covering authentication, menu browsing, order placement, order history, invalid submissions, and data persistence (**Figure 7**). Usability testing (**Figure 8)** was conducted with three participants using a task-based protocol. One user failed to recognise successful order submission due to limited

visibility of the confirmation message. In response, the interface was modified to automatically scroll to the confirmation message, improving feedback clarity and preventing repeated submissions.

3.3 Static Analysis and Performance Testing

Static code analysis was implemented using ESLint and enforced within the CI pipeline. Performance testing (**Figure 9**) was conducted manually by observing response times for core backend endpoints and frontend interactions under typical usage conditions, confirming acceptable responsiveness.

Section 4 - DevOps and Continuous Integration Pipeline

A continuous integration (CI) pipeline was implemented using GitHub Actions to automate build validation and testing throughout development. The pipeline is defined in `ci.yml` and is triggered automatically on pushes and pull requests to the main branch, as well as via manual dispatch. This ensures that changes merged into the primary branch are consistently verified, supporting early fault detection and helping to maintain a stable codebase. Docker was used locally to containerise the frontend and backend services, ensuring consistent runtime environments across development machines and reducing configuration drift between local development and CI execution.

The pipeline runs as a single build-and-test job on an `ubuntu-latest` GitHub-hosted runner, providing a clean and reproducible execution environment for each run. A single job was intentionally selected to reduce complexity and execution time, as the project is a student-scale application with closely related frontend and backend components. Node.js version 20 is installed using the official `actions/setup-node` action, aligning the CI environment with local development to minimise version-related inconsistencies. Dependency caching is enabled using npm and keyed against the backend and frontend `package-lock.json` files, improving performance by avoiding unnecessary reinstallation of unchanged dependencies.

The pipeline enforces a clear separation of concerns between backend and frontend validation. For the backend, dependencies are installed using `npm ci`, ensuring deterministic installs based on the lock file, followed by execution of automated unit and integration tests using `npm test`. This follows the “fail-fast” principle of CI, allowing regressions in authentication, order handling, or database interaction to be detected early.

For the frontend, dependencies are installed using `npm ci`, followed by static code analysis via `npm run lint -- --max-warnings=0`. This acts as a quality gate, causing the pipeline to fail if linting warnings are present and enforcing consistent coding standards. Finally, `npm run build` is executed to confirm that the frontend can be successfully compiled.

Concurrency control is enabled using a workflow group keyed to the Git reference, with in-progress runs cancelled when newer commits are pushed to the same branch. This reduces wasted compute resources and ensures feedback remains relevant. No deployment stage is configured, as the focus is continuous integration rather than delivery. Nevertheless, the pipeline demonstrates core DevOps principles, including automation, reproducibility, and rapid feedback, supporting reliable and incremental development.

Section 5 - Evaluation

The project delivered a technically sound system with reliable core functionality. The backend architecture worked well, with clear separation between routes, controllers, middleware, and data models supporting maintainability and testability. Automated testing and continuous integration contributed positively to system stability. However, introducing continuous integration later in development reduced its potential impact, and some development time was consumed by configuring testing and authentication frameworks rather than implementing additional features.

All planned functionality was successfully delivered. User authentication, menu browsing, order creation, and order history retrieval were implemented and operated reliably. Role-based access control supported different user responsibilities: staff users managed order status during service, while a manager role enabled authenticated management of menu categories, items, and pricing. While these roles aligned well with the original requirements, the system does not currently support administrative management of user accounts, which would be expected in a more complete production system.

The use of unit and integration testing proved highly valuable. Unit tests enabled isolated validation of security-critical logic such as authentication middleware, while integration tests verified correct interaction between API routes and the database. These techniques helped identify defects early and provided confidence during refactoring. Manual system and usability testing complemented automated tests by validating real user workflows. Continuous integration further reinforced these techniques by automating test execution and enforcing quality gates.

The chosen technologies were appropriate for the project scope. Node.js and Express supported rapid backend development, MongoDB and Mongoose enabled flexible data modelling, and React facilitated modular frontend design. ESLint and GitHub Actions were effective in maintaining code quality and consistency.

Key lessons include the importance of introducing continuous integration earlier, balancing feature scope against implementation effort, and planning user and role management carefully in multi-user systems. Comprehensive automated testing should be retained in future projects due to its clear impact on reliability and development confidence.

Section 6 – Appendix

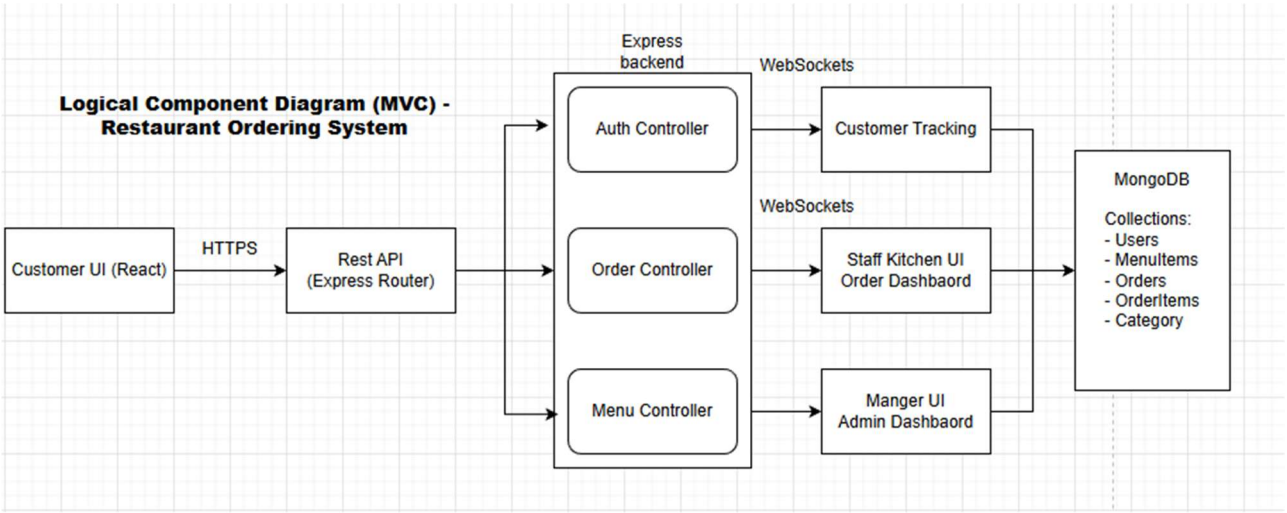

Figure 1: Component Diagram

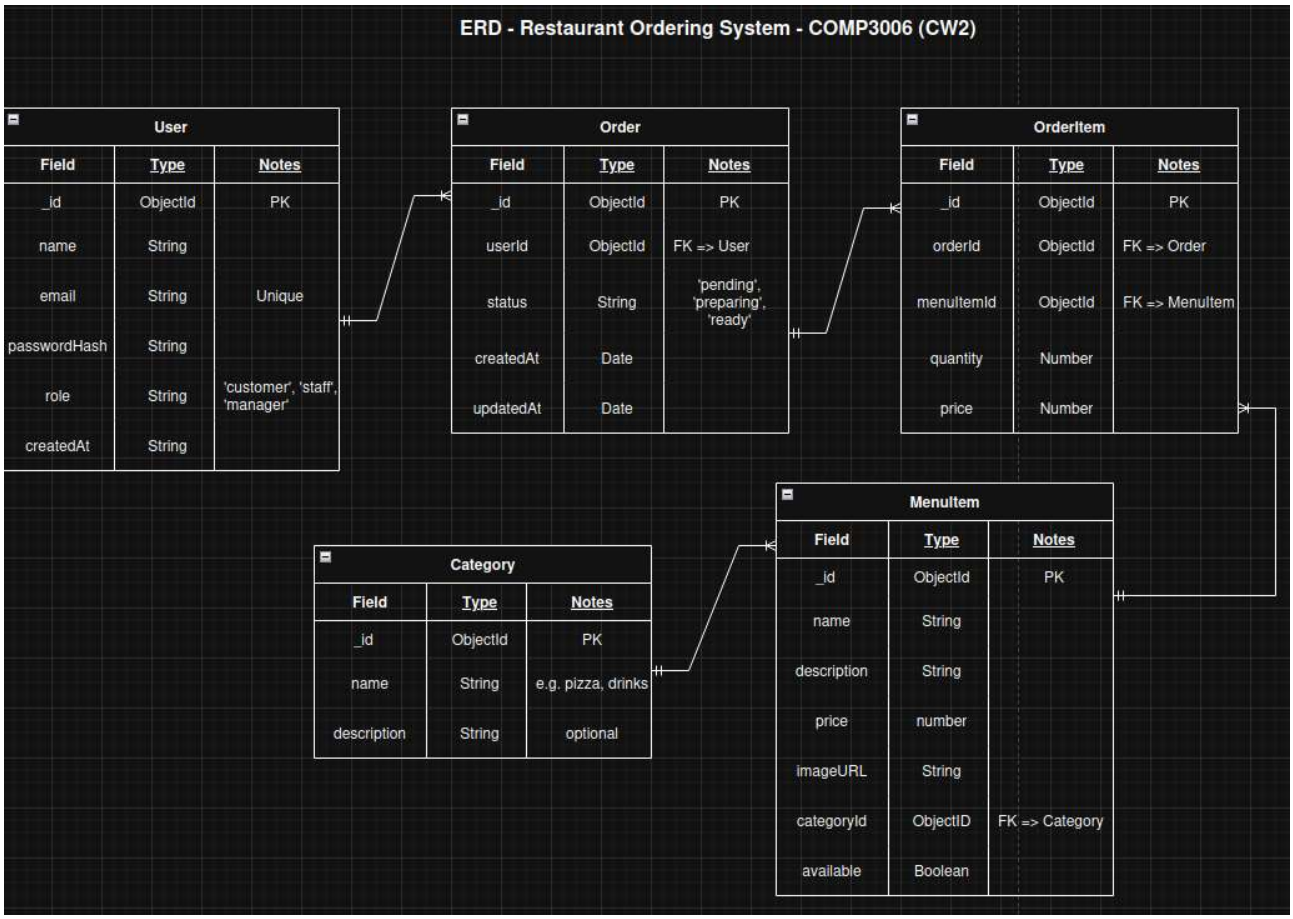

Figure 2: Entity relationship diagram (ERD)

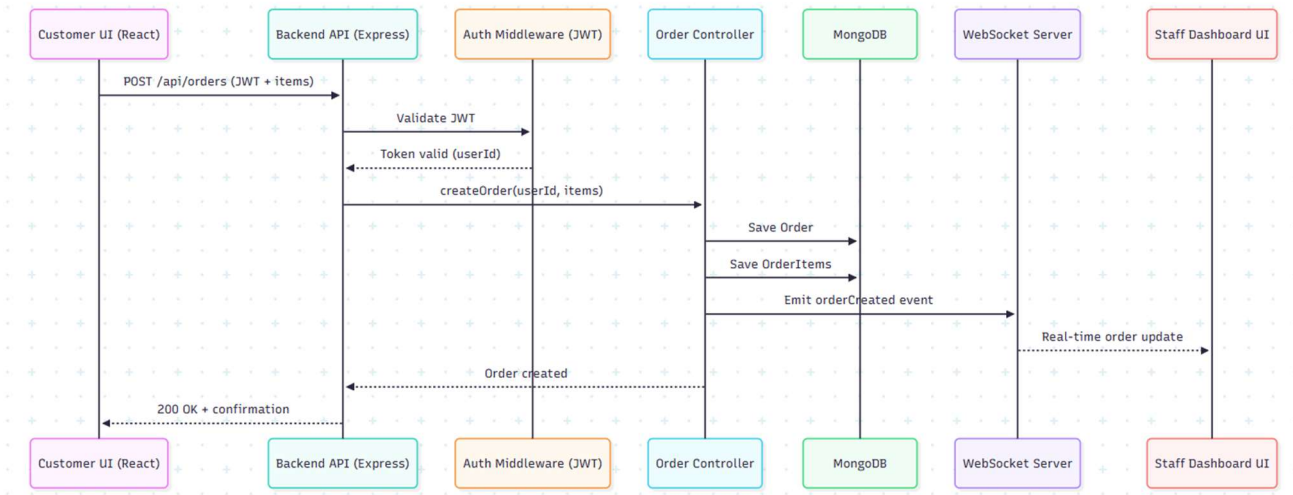

Figure 3: Sequence Diagram

Figure 4: authMiddleware.test.js

orders.api.test.js

backend > tests > integration > orders.api.test.js > ...

```
13  const buildApp = () => {
14    const app = express();
15    app.use(express.json());
16    app.use("/api/orders", orderRoutes);
17    return app;
18  };
19
20  describe("Orders API integration tests", () => {
21    const app = buildApp();
22    let token;
23    let userId;
24
25    beforeEach(async () => {
26      await mongoose.connection.dropDatabase();
27      process.env.JWT_SECRET = "testsecret";
28      const passwordHash = await bcrypt.hash("password123", 10);
29
30      const user = await User.create({
31        name: "Integration User",
32        email: "integration@test.com",
33        passwordHash
34      });
35
36      userId = user._id.toString();
37      token = jwt.sign({ userId }, process.env.JWT_SECRET);
38    });
39
40    test("POST /api/orders creates an order for authenticated user", async () => {
41      const category = await Category.create({
42        name: "Mains",
43        slug: "mains"
44      });
45      const menuItem = await MenuItem.create({
46        name: "Test Item",
47        price: 5,
48        categoryId: category._id
49      });
50      const res = await request(app)
51        .post("/api/orders")
52        .set("Authorization", `Bearer ${token}`)
53        .send({
54          items: [
55            { menuItemId: menuItem._id.toString(), quantity: 1 }
56          ]
57        });
58
59      expect(res.statusCode).toBe(200);
60
61      const orders = await Order.find();
62      const orderItems = await OrderItem.find();
63      expect(orders.length).toBe(1);
64      expect(orderItems.length).toBe(1);
65      expect(orders[0].userId.toString()).toBe(userId);
66    });
67
```

Figure 5: orders.api.test.js 1/2

```
68    test("GET /api/orders/user/:userId returns orders for that user", async () => {
69      const category = await Category.create({
70        name: "Starters",
71        slug: "starters"
72      });
73      const menuItem = await MenuItem.create({
74        name: "Test Item",
75        description: "Test description",
76        price: 5,
77        categoryId: category._id
78      });
79      const order = await Order.create({
80        userId
81      });
82      await OrderItem.create({
83        orderId: order._id,
84        menuItemId: menuItem._id,
85        quantity: 1,
86        price: 5
87      });
88
89      const res = await request(app)
90        .get(`/api/orders/user/${userId}`)
91        .set("Authorization", `Bearer ${token}`);
92
93      expect(res.statusCode).toBe(200);
94      expect(res.body.orders.length).toBe(1);
95      expect(res.body.orders[0].items.length).toBe(1);
96    });
97
98    test("GET /api/orders/user/:userId returns orders for that user without items", async () => {
99      await Order.create({
100       userId
101     });
102
103     const res = await request(app)
104       .get(`/api/orders/user/${userId}`)
105       .set("Authorization", `Bearer ${token}`);
106
107     expect(res.statusCode).toBe(200);
108     expect(res.body.orders.length).toBe(1);
109     expect(res.body.orders[0].items.length).toBe(0);
110   });
111
112   test("Unauthenticated request is rejected", async () => {
113     const res = await request(app)
114       .get(`/api/orders/user/${userId}`);
115
116     expect(res.statusCode).toBe(401);
117   });
118 });
```

Figure 6: orders.api.test.js 2/2

System Testing

Test ID	System Scenario	Preconditions	Steps Performed	Expected Result	Actual Result	Outcome (Pass/ Fail)
ST-01	User registration and login	Backend and frontend running	Create account → log in with valid credentials	User successfully authenticated and redirected	User logged in and redirected correctly	Pass
ST-02	Browse menu items	User logged in	Navigate to menu page	Menu categories and items displayed	Menu loaded correctly	Pass
ST-03	Place an order	User logged in, items available	Select items → submit order	Order created and confirmation shown	Order saved and confirmation displayed	Pass
ST-04	View order history	User has existing orders	Navigate to orders page	Previous orders displayed	Orders retrieved and shown correctly	Pass
ST-05	Authentication guard	User logged out	Attempt to access orders page	Redirect to login page	Redirected to login	Pass
ST-06	Submit empty order	User logged in	Attempt to submit order with no items	Validation error displayed	Error message shown	Pass
ST-07	Order persistence	Order previously placed	Refresh application and view orders	Order remains stored	Order persisted in database	Pass

Figure 7: System testing table

Usability Testing

User ID	User Background	Task	Steps Given	Outcome (Pass/Fail)	Observations	User Feedback	Improvement Identified
U-01	CS student, not involved in development	Sign up & Login	Create an account then login	Pass	No issues	"It wasn't particularly obvious that I successfully logged in"	Improve validation message for login process
U-01	CS student, not involved in development	Place an order	Browse menu, select items, submit order	Fail	Pressed submit order multiple times as order confirmation appears at the top of the screen.	"I wasn't sure that my order was confirmed"	Increase submit button prominence and prevent spam of post requests
U-01	CS student, not involved in development	View order history	Navigate to orders page	Pass	Expected orders to load automatically - required manual trigger	"I didn't expect to have to load the orders manually"	Load orders as page loads
U-02	Non-technical user	Sign up & Login	Create an account then login	Pass	Looked for a sign up button before clicking login button	"Simple sign-up process"	None
U-02	Non-technical user	Place an order	Select items and submit	Pass	No major issues	"Pretty straightforward"	None
U-02	Non-technical user	Submit empty order	Attempt to submit without items	Pass	Error message present but not obvious	"I couldn't place an empty order, as expected"	Improve validation message location
U-03	Non-technical user	View order history	Access previous orders	Pass	No issues	"I want my previous to include a price of the total order"	Improve order view by adding order totals

Figure 8: Usability testing table

Performance Testing

Test ID	Scenario	Component	Action Performed	Test Conditions	Observed Response Time	Expected Behaviour	Result	Notes
PT-01	Load menu data	Backend API	`GET /api/menuitems`	Local dev, ~16 items	~60 ms	Menu data returned promptly	Pass	No noticeable delay, slower response time at cold start (expected)
PT-02	Place order	Backend API	`POST /api/orders`	Authenticated user, 4 items	~140 ms	Order created successfully	Pass	Immediate response
PT-03	Retrieve order history	Backend API	`GET /api/orders/user/:id`	~5 existing orders	~80 ms	Orders returned correctly	Pass	Scales well at small dataset
PT-04	Menu page load	Frontend	Open menu page	Local browser	<1 second	Page renders smoothly	Pass	No UI lag
PT-05	Submit order	Frontend	Select items → submit	Standard workflow	Immediate feedback	Confirmation shown	Pass	Clear visual feedback, however, not always in view of the page - needs to appear relative to the screen.
PT-06	Orders page load	Frontend	Navigate to orders page	~5 orders	<1 second	Orders displayed correctly	Pass	Order retrieval relies on the user pressing 'load orders', this feature should be automatic

Figure 9: Performance testing table